

MIAE 215 · Comprehensive Topic Guide (Part 4)

Control Statements, Logic Flow, Flowcharts & Robotics Decision Systems

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Table of Contents

1. [Algorithmic Flowcharts & Visual Logic Modeling](#)
 2. [Selection Control Structures: Single `if` , Dual `if-else` , and Multi-Way Cascades](#)
 3. [Relational Comparison Operators & Dynamic State Evaluation](#)
 4. [The Assignment vs. Equality Cardinal Trap](#)
 5. [Floating-Point Decision Branching: The Epsilon \(\$\epsilon\$ \) Tolerance Architecture](#)
 6. [Boolean Logic Gates: AND \(`&&` \), OR \(`||` \), and Inversion \(`!` \)](#)
 7. [Short-Circuit Evaluation & Zero-Division Safety Guards](#)
 8. [Nested Branches, The Dangling `else` Ambiguity & Scope Braces](#)
 9. [Robotics Sensor & Actuator Decision Systems](#)
 10. [Teacher Lesson Code Walkthrough: `control\_statements1.cpp` & Assignment 1 Problems](#)
 11. [Week 3 In-Person Lecture Deep-Dive: Input Stream Processing, Negative Exit Guards & Double Accumulation](#)
 12. [Loop Statements: the for Loop, Exit Values & Nested Loops](#)
 13. [Operator Precedence & Evaluation Order Hierarchy \(Control Statements I Part 3\)](#)
 14. [Indefinite Iteration: `while` Loops & Robotics Infinite Executions \(Control Statements II Part 1\)](#)
 15. [Loop Control Alteration: `break` vs. `continue` & Sentinel Filtering](#)
-

1. Algorithmic Flowcharts & Visual Logic Modeling

Before writing a single line of C++ code, engineers design and verify algorithmic logic using **flowcharts**. A flowchart provides a formal graphic blueprint mapping decision forks, hardware states, and execution pathways.

Symbol	Shape	Meaning	Example
Terminal	Oval / capsule	Start or stop	Main, End
Input/output	Parallelogram	Stream input or output	<code>cin >> x</code> , Output
Process	Rectangle	Assignment or calculation	<code>x = 2*x + 1</code>
Decision	Diamond	A test with True/False exits	<code>x > 0?</code>

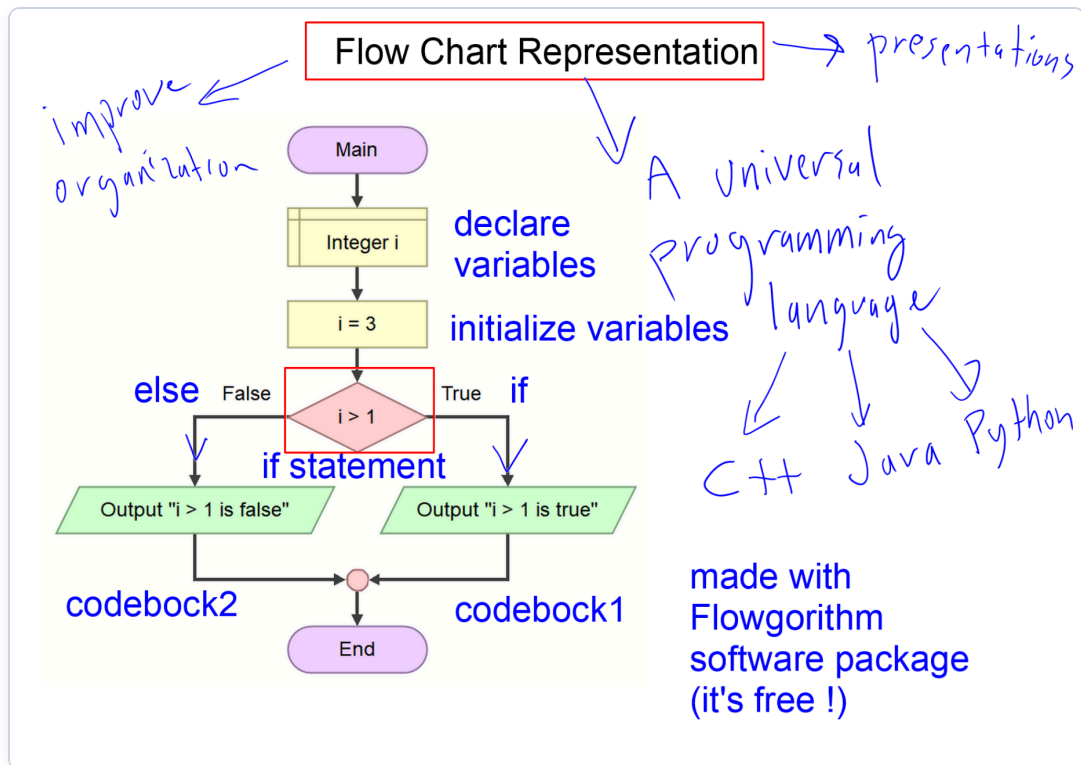


Figure 1: Flow chart representation, Control Statements I, p. 8 (with the teacher's annotations).

Reading the flowchart: it starts at **Main**, **declares variables** (`Integer i`), **initialises** them (`i = 3`), then reaches the diamond **if statement** (`i > 1`). The **False** branch runs

`codeblock2`

(Output "`i > 1` is false") and the **True** branch runs

`codeblock1`

; both rejoin before **End**. The teacher's notes explain why flowcharts matter: they "improve organization", are "a universal programming language" independent of C++, Java or Python, and are drawn with the free **Flowgorithm** package.

Flowgorithm in MIAE 215

Prof. Gordon provides **Flowgorithm** in `05 - Software & Flowcharts/` :

- `flow_chart1_simple_program.fprg` : Linear sequence execution, variable initialization, and output display.
- `flow_chart2_if_else.fprg` : Conditional decision forks representing robotic obstacle detection and threshold branching.

2. Selection Control Structures: Single `if` , Dual `if-else` , and Multi-Way Cascades

Selection statements allow a computer program to select between alternate code paths based on Boolean truth evaluations.

1. Single Selection: `if`

Executes an enclosed block of statements if and only if the test condition evaluates to `true` (any non-zero value):

```
if (temperature > 100.0) {  
    cout << "Warning: Reactor coolant exceeds boiling threshold!\n";  
}
```

2. Dual Selection: `if-else` (The "Fork in the Road")

Provides an alternative default execution path when the condition evaluates to `false` (zero):

```
if (distance_to_wall > 0.5) {  
    motor_forward();  
} else {  
    motor_stop(); // Default safe action  
}
```

3. Multi-Way Selection: Cascading `else if`

Evaluates a sequence of mutually exclusive conditions from top to bottom. Execution halts at the first matching branch:

```
if (score >= 90.0) {  
    grade = 'A';  
} else if (score >= 80.0) {  
    grade = 'B';  
} else if (score >= 70.0) {  
    grade = 'C';  
} else {  
    grade = 'F';  
}
```

if - else ladders

It's possible to have one if statement inside of another (ie "nested"), for example:

```
if ( i == 1 ) {  
    cout << "\ni == 1";  
} else { ←  
    if ( i == 2 ) {  
        cout << "\ni == 2";  
    } else { ←  
        if ( i == 3 ) {  
            cout << "\ni == 3";  
        } else { ←  
            cout << "\ni is not equal to 1, 2, 3 -- none of the above";  
        } ←  
    } ←  
} ←
```

Figure 3: if-else ladders, *Control Statements I (part 2)*, p. 2 (with the teacher's annotations).

Reading the slide: the nested form (an `if` inside each `else`) is drawn with arrows marking each `{ }` pair. It is exactly the same logic as the flat `else if` ladder. The tests run **top to bottom**, the first true one executes its block, and control jumps past the rest; the final `else` is the "none of the above" default.

3. Relational Comparison Operators & Dynamic State Evaluation

Test Conditions

The comparison operators below can be used in test conditions:

greater than, less than: $>$, $<$

greater than or equal, less than or equal: $>=$, $<=$

equal, not equal: $==$, $!=$

```
if( i >= k ) cout << "\ni is greater than or equal to k";
```

```
if( i == k ) cout << "\ni is equal to k";
```

equality conditions should normally not be used with float / double variables -- round off error makes execution unlikely, eg

```
// if statement is executed only if x = 0.000000000000000000e0
```

```
if( x == 0.0 ) cout << "\nx is equal to zero";
```

thus the if statement might fail even if x is very small and equal to zero for engineering purposes

Figure 4: Test conditions, Control Statements I, p. 2.

Reading the slide: the comparison operators are $>$, $<$, $>=$, $<=$, $==$ (equal) and $!=$ (not equal). The warning at the bottom is the key one: **equality tests should normally not be used with float/double**, because round-off makes `x == 0.0` fail when `x` is really `1e-20`. That is why Section 5 uses an epsilon tolerance.

Relational operators compare two operands and produce a Boolean result: `true` (1) or `false` (0).

Operator	Mathematical Meaning	True Example	False Example
$>$	Greater Than	<code>5 > 3</code> $\implies$ 1	<code>3 > 5</code> $\implies$ 0
$<$	Less Than	<code>2 < 8</code> $\implies$ 1	<code>8 < 2</code> $\implies$ 0
$>=$	Greater Than or Equal To	<code>4 >= 4</code> $\implies$ 1	<code>3 >= 4</code> $\implies$ 0
$<=$	Less Than or Equal To	<code>5 <= 7</code> $\implies$ 1	<code>9 <= 7</code> $\implies$ 0
$==$	Exactly Equal To	<code>10 == 10</code> $\implies$ 1	<code>10 == 11</code> $\implies$ 0
$!=$	Not Equal To	<code>7 != 9</code> $\implies$ 1	<code>7 != 7</code> $\implies$ 0

Dynamic Variable Re-Evaluation (Teacher Demonstration)

In `control_statements1.cpp`, Prof. Gordon emphasizes that condition evaluations depend on the **exact line of execution** in RAM:

```
int i = 1, k = 2;

if (k > i) {
    cout << "\nk is greater than i"; // EXECUTES (2 > 1 is true)
}

i = 3; // Variable state in RAM changes!

if (k > i) {
    cout << "\nk is greater than i"; // DOES NOT EXECUTE (2 > 3 is false)
}
```

4. The Assignment vs. Equality Cardinal Trap

By far the most catastrophic, silent bug in C++ programming is confusing the **assignment operator** (`=`) with the **relational equality operator** (`==`).

	<code>if (x == 5)</code> (intended comparison)	<code>if (x = 5)</code> (accidental assignment)
What happens to <code>x</code>	Read only	Overwritten with 5
Value of the condition	True or false depending on <code>x</code>	5, which is non-zero, so always true
Result	Correct test	Silent bug: the if-block always runs

```
int pressure = 0; // Safe pressure

if (pressure = 100) { // BUG: Assigns 100 to pressure! 100 is non-zero (TRUE)!
    emergency_shutdown(); // ALWAYS EXECUTES, even when pressure was 0!
}
```

Teacher Exam Warning / Pitfall:

The statement `if (x = 5)` is **syntactically legal C++**! The compiler will NOT produce an error. It overwrites `x` with `5`, and because `5` is non-zero, the `if` block executes every single time. Always double-check for `==` in decision conditions!

5. Floating-Point Decision Branching: The Epsilon (ϵ) Tolerance Architecture

As demonstrated in Part 1 and Part 3, binary floating-point representation cannot represent decimal fractions exactly.

The Round-Off Trap in `if` Statements

In `control_statements1.cpp`, Prof. Gordon presents:

```
double x = 1.0e-30;

// NAIVE CONDITION:
if (x == 0.0) {
    cout << "x is zero"; // WILL NEVER EXECUTE!
}
```

Although 10^{-30} is physically zero for all engineering measurements, `x == 0.0` requires every single one of the 64 bits to be zero.

The Epsilon Tolerance Approach

Define a small tolerance constant ϵ (e.g., 10^{-9}):

```
double eps = 1.0e-9;

// Checking if x is approximately zero:
if (abs(x) < eps) {
    cout << "x is zero within engineering tolerance\n";
}

// Checking if x is approximately equal to a target (e.g., 5.5):
if (abs(x - 5.5) < eps) {
    cout << "x is approximately equal to 5.5\n";
}

// Checking if x is approximately less than or equal to 5.5:
if ((x < 5.5) || (abs(x - 5.5) < eps)) {
    cout << "x is approximately <= 5.5\n";
}
```

6. Boolean Logic Gates: AND (`&&`), OR (`||`), and Inversion (`!`)

To evaluate complex engineering states (e.g., "altitude $>$ 1000 AND velocity $<$ 250"), simple relational comparisons are joined using Boolean logical operators.

Operator	Name	True when
<code>&&</code>	Logical AND	Both conditions are true
<code> </code>	Logical OR	At least one condition is true
<code>!</code>	Logical NOT	The condition is false (inverts it)

Logical Operators

comparison operators can be combined using logical operators for more complex test conditions in if statements, eg

```
if( (i > k) && (k <= 3) ) {  
    cout << "(i is greater than k) AND (k is less than or equal to 3)";  
}
```

The following logical operators are available in C++:

AND: &&

- both conditions have to be true for the result to be true

OR: ||

- only one of the conditions have to true for the result to be true

Figure 2: Logical operators, Control Statements I, p. 4.

Reading the slide: comparisons are combined with `&&` and `||` for more complex tests, e.g. `if( (i > k) && (k <= 3) )` prints "i is greater than k AND k is less than or equal to 3". Multiple ANDs and ORs are allowed.

Comprehensive Truth Table

Condition <i>A</i>	Condition <i>B</i>	$A \text{ \&\& } B$	$A B$	$!A$
false (0)	false (0)	false (0)	false (0)	true (1)
false (0)	true (1)	false (0)	true (1)	true (1)
true (1)	false (0)	false (0)	true (1)	false (0)
true (1)	true (1)	true (1)	true (1)	false (0)

```
int i = 1, k = 2;  
  
// Multiple logical conditions chained together:  
if ((i < k) && (k <= 3) && (i < 10)) {  
    cout << "All three physical conditions are simultaneously satisfied.\n";  
}
```

7. Short-Circuit Evaluation & Zero-Division Safety Guards

C++ evaluates logical expressions **strictly from left to right** and employs **short-circuit evaluation**:

- In `A && B`: If `A` evaluates to `false`, the entire expression cannot possibly be true. ***B* is never evaluated!**
- In `A || B`: If `A` evaluates to `true`, the entire expression is already guaranteed true. ***B* is never evaluated!**

Guarding Against Division-by-Zero

This hardware guarantee allows engineers to write foolproof safety guards in a single line:

```
double denominator = 0.0;
double numerator = 100.0;

// SHORT-CIRCUIT SAFETY GUARD:
if (denominator != 0.0 && (numerator / denominator > 10.0)) {
    cout << "Threshold exceeded!\n";
}
```

Because `denominator != 0.0` evaluates to `false`, the CPU **halts evaluation immediately**. The division `numerator / denominator` is never executed, preventing a fatal hardware division-by-zero crash!

8. Nested Branches, The Dangling `else` Ambiguity & Scope Braces

When nesting `if` and `if-else` blocks without explicit braces `{ }`, the C++ compiler associates an `else` clause with the **most immediate preceding unclosed `if`**.

```
// DANGEROUS MISLEADING INDENTATION:
if (speed > 0)
    if (speed > 100)
        cout << "Over speeding!";
else
    cout << "Vehicle is stationary?"; // LOGIC ERROR: Binds to (speed > 100), NOT (speed > 0)!
```

The Professional Engineering Rule: Always Use Braces

```
// ROBUST, AMBIGUITY-FREE CODE:
if (speed > 0) {
    if (speed > 100) {
        cout << "Over speeding!";
    }
} else {
    cout << "Vehicle is stationary."; // Clearly binds to (speed > 0)
}
```

9. Robotics Sensor & Actuator Decision Systems

In `control_statements1.cpp`, Prof. Gordon connects control logic to mechanical automation:

```

#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    double robot_distance;

    // Simulate reading from ultrasonic or LIDAR sensor
    cout << "Enter robot distance from obstacle (meters): ";
    cin >> robot_distance;

    if (robot_distance > 0.5) {
        // Send PWM forward voltage to DC motors
        cout << "Path clear. Actuating motors FORWARD.\n";
    } else {
        // Obstacle detected within 0.5m: engage emergency brake
        cout << "Obstacle imminent! Halting motors and engaging BRAKE.\n";
    }

    // Two getchar() calls to handle leftover stream newline
    getchar();
    getchar();

    return 0;
}

```

10. Teacher Lesson Code Walkthrough: `control_statements1.cpp` & Assignment 1 Problems

Assignment 1, Question 5: Threshold Detection with Sentinel Break

Problem: Calculate $f(t) = \sin(t)$ for $t = 0.0, 0.1, 0.2 \dots 10^7$. Terminate the loop immediately when $0.9 < f(t) < 0.93$ using a single `if` statement.

```

#include <iostream>
#include <stdio>
#include <cmath>

using namespace std;

int main() {
    double f, t;

    for (t = 0.0; t < 1.0e7 + 0.1; t += 0.1) {
        f = sin(t);

        // Single if statement utilizing logical AND (&&)
        if ((f > 0.9) && (f < 0.93)) {
            break; // Immediately exit loop
        }
    }

    cout.precision(6);
    cout << "Loop terminated at t = " << t << " s\n";
    cout << "Function value    f = " << f << "\n";

    return 0;
}

```

Assignment 1, Question 6: Numerical Halving & Oscillatory Convergence

Problem: Input x . Repeatedly divide x by 2 until both $x < 1.0 \times 10^{-7}$ AND $\sin(1/x) > 0.9$. Print final values in scientific notation with 10 digits precision.

```
#include <iostream>
#include <stdio>
#include <cmath>

using namespace std;

int main() {
    int i;
    double x;

    cout << "Input initial x value: ";
    cin >> x;

    for (i = 0; i < 1000000; i++) {
        // Compound logical break condition
        if ((x < 1.0e-7) && (sin(1.0 / x) > 0.9)) {
            break;
        }
        x = x / 2.0; // Halve search interval
    }

    cout << scientific;
    cout.precision(10);

    cout << "Final x          = " << x << "\n";
    cout << "sin(1.0 / x)       = " << sin(1.0 / x) << "\n";
    cout << "Total Halvings    = " << i << "\n";

    return 0;
}
```

Execution State Trace for Question 6

Iteration (<i>i</i>)	Current <i>x</i>	$x < 1.0 \times 10^{-7}$	$\sin(1/x)$	$\sin(1/x) > 0.9$	Combined Break (&&)	Next <i>x</i>
0	1.000	false	0.841470	false	false	0.500
1	0.500	false	0.909297	true	false (x too large)	0.250
...	...	...	...	...	...	...
24	5.960×10^{-8}	true	-0.342110	false	false	2.980×10^{-8}
Convergence	$x < 10^{-7}$	true	> 0.900000	true	true (Loop Breaks)	Final <i>x</i> output

11. Week 3 In-Person Lecture Deep-Dive: Input Stream Processing, Negative Exit Guards & Double Accumulation

In the **Week 3 In-Person Lecture**, Prof. Gordon introduced interactive stream input processing with loop counters, defensive validation guards, and immediate program abortion strategies.

Lecture Problem Statement

Write a robust C++ engineering program that satisfies two strict operational constraints:

- 1. **Part (a) Threshold Accumulation:** Prompt the user to input 5 floating-point numbers (`double`) from the keyboard one at a time. Count how many of these doubles have values strictly less than `7.7` (`value < 7.7`). Display the final count upon normal loop completion.
- 2. **Part (b) Immediate Safety Exit Guard:** If at any point the user inputs a negative number (`value < 0.0`), immediately terminate program execution using either `return 1;` (from `main`) or `exit(1);` (from `<cstdlib>`). No further inputs must be accepted and the summary report must not be printed.

Defensive Exit Strategies: `exit(status)` vs. `return code`

	<code>return code;</code>	<code>exit(status_code);</code>
Defined in	The language core	<code><cstdlib></code>
Effect	Returns from the current function ; inside <code>main()</code> it ends the program and returns the code to the OS	Terminates the whole process from any depth
Cleanup	Destroys local objects on the way out	Flushes buffers, runs <code>atexit</code> handlers

In engineering control software, entering abnormal states (e.g., negative pressures, negative Kelvin temperatures, or invalid sensor inputs) often demands an immediate failsafe shutdown.

Complete Production Implementation (w3_l1_in_person_example1_count_doubles.cpp)

```
#include <iostream>
#include <stdio>
#include <cmath>
#include <cstdlib> // Required for exit()

using namespace std;

int main() {
    const int TOTAL_INPUTS = 5;
    const double THRESHOLD = 7.7;
    int count_less_than_threshold = 0;
    double current_value = 0.0;

    cout << "=== MIAE 215: Week 3 In-Person Lecture 1 Example ===\n";
    cout << "Enter " << TOTAL_INPUTS << " floating-point numbers one at a time.\n";
    cout << "(Note: Entering any negative value terminates the program immediately)\n\n";

    for (int i = 1; i <= TOTAL_INPUTS; i++) {
        cout << "Enter double #" << i << ": ";
        cin >> current_value;

        // Part (b): Immediate negative input guard
        if (current_value < 0.0) {
            cout << "\n[ALERT] Negative value detected (" << current_value << ")\n";
            cout << "Stopping program immediately via exit(1)...\n";
            exit(1); // Immediate process termination
        }

        // Part (a): Count values strictly less than 7.7
        if (current_value < THRESHOLD) {
            count_less_than_threshold++;
        }
    }

    cout << "\n=====\n";
    cout << "Total numbers evaluated: " << TOTAL_INPUTS << "\n";
    cout << "Numbers with value < " << THRESHOLD << ": " << count_less_than_threshold << "\n";
    cout << "=====\n";
    cout << "\nDone.\n";

    return 0;
}
```

Execution Trace Analysis

Scenario A: Nominal Execution (All Non-Negative Numbers)

User Inputs: 3.2 , 8.4 , 7.7 , 1.5 , 9.0

- `3.2 < 7.7` $\implies$ Count = 1
- `8.4 < 7.7` $\implies$ False (Count = 1)
- `7.7 < 7.7` $\implies$ False (Strict inequality! Count = 1)
- `1.5 < 7.7` $\implies$ Count = 2
- `9.0 < 7.7` $\implies$ False (Count = 2)
- **Output:** Total numbers evaluated = 5, Numbers < 7.7 = 2. Exit Code = 0.

Scenario B: Premature Termination (Negative Value Entered on Step 3)

User Inputs: `4.5`, `6.1`, `-0.05`

- Step 1 (`4.5`): `< 7.7` (Count = 1)
- Step 2 (`6.1`): `< 7.7` (Count = 2)
- Step 3 (`-0.05`): `current_value < 0.0` is `true`.
- **Action:** Alert printed, `exit(1)` executed immediately. Remaining 2 inputs are bypassed. Final summary block is NEVER executed. Exit Code = 1.

Floating-Point Comparison Caveats Revisited

During this lecture, Prof. Gordon reinforced why `current_value == 0.0` or direct equality checks on `double` types are hazardous:

- Due to binary representation limitations (IEEE 754), a calculation producing a theoretical zero may evaluate to `1.0e-17` or `1.0e-30`.
- Direct equality `x == 0.0` evaluates to `false`.
- Always use relational inequalities (`<`, `>`, `<=`, `>=`) or epsilon tolerance bands: `fabs(a - b) < 1.0e-7`.

12. Loop Statements: the for Loop, Exit Values & Nested Loops

A **loop** executes part of the program repeatedly (Control Statements I part 2, p. 5). The `for` loop is the most common kind.

Loop Statements

A loop is used to execute a part of the program repeatedly

The for loop is the most common type of loop, eg

0. initialize index variable 1. test condition 4)
3. index update 6)
i=0,1,2,3,4
for(i = 0 ; i < 5 ; i++) {
 cout << "\ni = " << i; 2. excute codeblock 5)
}
i=5
the loop repeats (steps 1 to 3) until the test condition is false
note that the {} is not needed for single line code blocks, eg
for(i = 0; i < 5; i++) cout << "\ni = " << i;

Figure 5: Loop statements, Control Statements I (part 2), p. 5 (with the teacher's annotations).

Reading the slide: the order in which the parts run.

Step	Part of <code>for(i = 0 ; i < 5 ; i++)</code>	When it runs
0	<code>i = 0</code> : initialise the index	Once, before anything else
1	<code>i < 5</code> : test the condition	Before every pass; if false, the loop ends
2	The code block <code>cout << "\ni = " << i;</code>	Only when the test was true
3	<code>i++</code> : update the index	After every pass, then back to step 1

The teacher's notes list the printed values `i = 0, 1, 2, 3, 4`, and underneath the closing brace, `i = 5`: the update runs one last time, the test `5 < 5` fails, and the loop exits with `i` equal to 5. For a single-statement body the braces are optional.

Other update expressions (p. 6)

The update does not have to be `i++`. Trace each one by listing the values that pass the test:

Loop	Values printed	Exit value
<code>for(i = 0; i <= 10; i += 2)</code>	0, 2, 4, 6, 8, 10	<code>i = 12</code>
<code>for(x = 1.0; x < 1.0e5; x *= 10.0)</code>	1, 10, 100, 1000, 10000	<code>x = 1.0e5</code>
<code>for(k = 0; k < 1000; k = k*k + 1)</code>	0, 1, 2, 5, 26, 677	<code>k = 458330</code>

The **exit value** is always the first value that

fails

the test, never the last one printed, because the update runs before the failing test. This is the most common trap in "what is printed after the loop?" questions.

Changing the index inside the body changes the trace: in `for( i = -1; i <= 5; i = i + 2 ) { cout << i; if( i == 3 ) i = 7; }` the values -1, 1, 3 print; at 3 the body sets `i = 7`, the update makes it 9, and the test `9 <= 5` ends the loop.

Nested loops (p. 7)

it's possible to put a for loop inside a for loop (ie a nested loop)

this is very useful for 2D arrays, matrices, etc. eg,

```
double A[3][3];
```

```
for(i=0;i<3;i++) { // outer loop (i = row)
    // for each i this inner loop goes over all j
    for(j=0;j<3;j++) { // inner loop (j = col)
        cout << "\ni = " << i << " , j = " << j;
        A[i][j] = 1.0 + i + j; // initialize A
    }
}
```

therefore, this example sets a 2D array one row at a time

Figure 6: Nested loops, Control Statements I (part 2), p. 7.

Reading the slide: the outer loop (`i`, the row) runs 3 times; for **each** value of `i`, the inner loop (`j`, the column) runs 3 times. So the inner body runs $3 \times 3 = 9$ times, and it fills the 2D array one row at a time with `A[i][j] = 1.0 + i + j` (for example `A[2][1] = 1 + 2 + 1 = 4`). In general, the inner body runs (outer count) $\times$ (inner count) times.

13. Operator Precedence & Evaluation Order Hierarchy (Control Statements I Part 3)

When evaluating compound boolean expressions, C++ follows an unambiguous operator precedence hierarchy analogous to mathematical arithmetic order (*BEDMAS*).

The Evaluation Hierarchy

Highest Precedence	-----
1. ()	Parentheses (force grouping and override precedence)
2. !, +, -	Unary NOT, unary positive, unary negative
3. *, /, %	Multiplicative arithmetic
4. +, -	Additive arithmetic
5. <, <=, >, >=	Relational comparison
6. ==, !=	Equality / inequality
7. &&	Logical AND (left-to-right, short-circuits)
8.	Logical OR (left-to-right, short-circuits)
9. =, +=, -=, *=, /=	Assignment operators (right-to-left)
Lowest Precedence	-----

Precedence Tracing Rules & Examples

Given variables: `i = 3`, `k = 1`, `x = 1.1`:

1. Arithmetic before Relational before Logical OR:

```
if ( i + k > 0 || i < k ) // Step 1: 3 + 1 = 4; Step 2: 4 > 0 (true); Step 3: short-circuit -> true
```

2. Unary NOT (!) binds tightly:

```
if ( k > 0 || !(i < k) ) // Step 1: k > 0 (true); short-circuit -> true
```

3. && binds before || :

```
if ( i > 0 && i < k || i < 7 )
// Evaluates as: ((i > 0) && (i < k)) || (i < 7)
// (true && false) || true ==> false || true ==> true
```

4. Logical Expressions as Integers: In C++, `0` represents `false` and any non-zero integer represents `true`. Logical operators evaluate to integer `1` (`true`) or integer `0` (`false`):

```
int i1 = 1, i2 = 0, i3;
bool b1 = true, b2 = false;
i3 = i2 || i1 + 1 && b1 && b2;
// 1. i1 + 1 = 2 (non-zero => true)
// 2. 2 && true && false => false (0)
// 3. 0 || 0 => 0
// Result: i3 = 0
```

14. Indefinite Iteration: `while` Loops & Robotics Infinite Executions (Control Statements II Part 1)

While `for` loops are optimal when the number of iterations n is known

a priori

, `while` loops govern **indefinite iteration**—where execution continues until an external event, user input, or sensor threshold is reached.

Structure of a `while` Loop

```
int i = 1; // Manual initialization required before loop header!
while( i < 10 ) {
    cout << "\ni = " << i;
    i++;    // Index update MUST occur inside the body!
}
// Exit state: i = 10 immediately upon loop termination
```

Infinite Control Loops in Mechatronics & Robotics

In embedded microcontrollers (such as Arduino systems controlling mobile robots), execution never exits. The controller continuously loops to read sensors and actuate motors:

```
while ( 1 ) {
    // 1. Sample ultrasonic sensor
    // 2. Compute error e(t)
    // 3. Drive PWM motor H-bridge
}
```

15. Loop Control Alteration: `break` vs. `continue` & Sentinel Filtering

C++ provides two jump statements that alter standard loop iteration flow:

1. The `break` Statement (Immediate Exit)

Terminates the innermost enclosing loop immediately and transfers control to the first statement following the loop:

```
while( 1 ) {
    cin >> k;
    if( k < 0 ) break; // Immediately breaks out; does NOT execute downstream lines
    if( k > 7 ) count++;
}
```

Why `break` is Superior to Index Mutation (`i = nmax`):

Setting `i = nmax` inside a `for` loop body does **not** stop the current iteration immediately—any subsequent lines in the loop body still run on the invalid data. `break` stops immediately, ensuring defensive data isolation.

2. The `continue` Statement (Iteration Skip)

Skips the remaining statements in the **current pass** and jumps directly to the loop update/condition:

```
for(int i = 1; i <= 10; i++) {  
    if( i > 5 ) continue; // For i = 6..10, skips the line below  
    cout << " loop";  
}
```

[!WARNING] In a `while` loop, executing `continue` before updating the loop index variable will cause an **accidental infinite loop**, because the update statement is skipped! Always place index increments before `continue` in `while` structures.